Predicting Sprint Trouble Without Hindsight: A Benchmark of Feature Families on 29 Public Jira Projects

Hani AbuSharkh*

November 7, 2026

Abstract

Sprint-risk models are usually trained on a present-day Jira export, which shows how a sprint ended rather than what the team knew when it began, and each study reports its own features on its own data. We state formally what a feature may use at prediction time and derive four consequences: empty sprints inflate ROC-AUC by an exact amount, a fitted threshold carries a hidden cost ratio, a binomial model says which spillover label team history should help with, and a beta-binomial extension says how far that model's probabilities can be trusted. We then score six feature families on the same 4,235 sprints from 29 TAWOS projects under one time-ordered protocol, for late closure, spillover and three new labels. Progress counts dominate during the sprint, and burndown-shape features add nothing to them. Team history adds 0.05 to 0.07 AUC on top of progress counts for the spillover labels and little for lateness. Four new planning-time features add about 0.03 AUC for the two spillover labels. Serious spillover is the most useful new label: history beats scope by 0.10 AUC, which the binomial model predicted and our own hypothesis did not. A binomial predictor with one estimated number per team and no learning comes within 0.03 AUC of gradient boosting, but its probabilities are too extreme, because the issues of a sprint do not finish independently. On 21 projects that played no part in forming the hypotheses, the in-sprint and serious-spillover results replicate. On ten outside projects, none of the history gains transfer.

Index Terms— *Agile software development, data leakage, Jira, machine learning, mining software repositories, Scrum, sprint risk, ROC analysis*

1 Introduction

A Scrum team commits to a set of issues at the start of each sprint. Some sprints end with work left over, and some are closed in Jira days after their planned end. Both are visible to everyone once the sprint is over. The useful question is whether they can be seen coming, at planning or early in the sprint, from what the tracker already records.

*H. AbuSharkh is with the Higher Colleges of Technology, Al Ain, UAE (e-mail: habusharkh@hct.ac.ae).

Several studies have tried [1–5], and industrial work has studied what makes delivery late [6]. Two things make their results hard to use. The first is hindsight. A Jira export is a snapshot taken years later: an issue's sprint field, story points and status are the final values unless the change log is replayed. A model trained on the snapshot can learn from facts that did not exist at planning, and its accuracy then says little about what a team would see in practice [7, 8]. The second is that every study brings its own features, labels, data and protocol. If one paper reports a good AUC with burndown features and another with iteration counts, nobody can tell which kind of signal does the work, because the two numbers were never measured on the same sprints.

This paper takes both problems in turn. Our earlier work [9] rebuilt the board as it stood on the first day of each sprint for eight TAWOS projects [10], and showed how much two common mistakes inflate the results. Here we turn the same reconstruction into a benchmark. We extend it to every TAWOS project with enough sprints (29 projects, 4,235 sprints), state the no-hindsight rule formally, and score six feature families side by side on the same sprints, the same labels and the same split.

The contributions are as follows.

1. *A formal setting with checkable consequences* (Section 3). We define an admissible feature as one that depends only on events logged before the prediction time, and derive four results: an exact decomposition of how empty sprints inflate ROC-AUC, the cost ratio a Youden threshold silently assumes, a binomial model that says which spillover label history should help with, and a beta-binomial extension that says when that model's probabilities can be trusted. Each result is checked against our data, and the binomial model is also scored as a predictor with no learning.

2. *A head-to-head benchmark* (Sections 4 to 6). Committed scope, planning-time team history, four new planning-time features, in-sprint progress counts, burndown-shape features after Pérez Castillo et al. [4], and the iteration features of Choetkiertikul et al. [1] that a public export supports, all scored under one protocol with one stated primary analysis and two kinds of bootstrap interval.

3. *Three new outcome labels with a verdict on each*: se-

rious spillover, seriously late and scope churn. We report which ones a model can rank and which ones look like team habit.

4. *A confirmation set and an outside test.* We repeat every hypothesis test on the 21 projects that played no part in writing the hypotheses, and test on ten projects from two organisations that are not in TAWOS, which shows where the gains stop.

The short answer is that progress counts carry most of the in-sprint signal, burndown shape adds nothing, team history helps with how much is left over but not with when the sprint closes, and none of the history gains survive a move to another organisation. At planning time, most of what a learned model does is estimate one number per team: a binomial predictor built on the team's recent completion rate nearly matches it. We wrote our hypotheses down before the full run (Section 5) and report where they failed. One failed in an instructive way: we expected history to add nothing to serious spillover, and the theory in Section 3 predicted the opposite. The theory was right.

2 Related Work

2.1 Predicting sprint and iteration outcomes

Choetkiertikul et al. [1] predicted how much of an iteration's committed work would be delivered, using iteration-level counts (issues and points at start, added, removed, done, to do) together with issue-level and team features, on open-source projects. The same group predicted risks and delays at the level of single issues [11, 12], and Sprint2Vec [2] learns a sprint embedding from issue text and developer activity. Hearty et al. [3] modelled velocity in XP with a dynamic Bayesian network. In industry, Kula et al. studied the factors behind on-time delivery in large-scale agile development [6] and predicted delays of user stories from team dynamics [13]. More recently, Pérez Castillo et al. [4] graded finished sprints from eleven burndown-shape indicators (R1 to R11) computed in their Worki tool, and Obike et al. [5] scored sprint deliverability with a weighted index. Effort estimation [14–17] is a neighbouring problem: it predicts the size of an issue, not the fate of a sprint.

All of this work rests on issue-tracker data, and public Jira datasets [10, 18, 19] have made it reproducible. They are, however, exports taken long after the fact. The studies above differ in data, labels and validation, and in how strictly features are limited to what was known at prediction time. In cross-validation over iterations, a training fold can hold iterations that happened after the test ones [1]. No study compares the feature families on common data. We do not claim to reproduce any of them. We implement the parts of two feature families that a public export supports and score them on our sprints, which measures the families, not the papers.

2.2 Leakage and time-aware evaluation

Leakage, where a model sees information it would not have when deployed, is a known cause of inflated results [7, 8]. In software analytics, the choice of validation technique changes the performance a defect model appears to have [20], random splits mix past and future [21, 22], and labels that are only known later leak into features [23]. Shepperd et al. [24] found that which research group ran a defect-prediction study explained more of the variation in reported performance than the choice of learner. A common protocol on common data is the remedy we apply here. Our earlier work [9] showed, on eight TAWOS projects, that two reproduced leakage mistakes inflated F1 by up to 0.24 and that shuffling sprints before splitting added 0.07 AUC. The present paper reuses that reconstruction but none of its results.

2.3 Predictive monitoring and warnings as decisions

Predictive process monitoring [25, 26] faces the same problem with event logs: it predicts the outcome of a running case from a prefix of its events, and benchmarks compare methods on common logs [26, 27]. Our in-sprint checkpoints are prefixes in this sense. Alarm-based monitoring [28] treats a warning as a decision with a cost, which is the view we take in Section 3.

2.4 Ranking, thresholds and calibration

ROC-AUC measures ranking [29]. It says nothing about where to set a threshold, and its use for comparing classifiers has known limits [30, 31]. A cost-optimal threshold for a calibrated score follows from the costs alone [32], which is why calibration matters. The Brier score [33] and the calibration slope [34] measure it, and boosted trees are known to produce distorted probabilities unless checked [35].

3 Formal Setting

This section fixes what "no hindsight" means for a Jira export and derives four consequences. Each one turns a pattern that is easy to miss into a prediction the experiments can check: how empty sprints inflate ROC-AUC, why a threshold fitted on one group of teams does not carry to another, how the two spillover labels should differ, and why a simple model of sprint completion can rank sprints well yet give probabilities that are too extreme.

3.1 The event log and the sprint state

A Jira export is an event log $\mathcal{E} = \{(i, f, v^-, v^+, t)\}$: issue i had field f changed from v^- to v^+ at time t. With each issue's creation time γ_i and resolution time r_i (undefined while the issue is open), the log gives the value of any

field at any time by replaying the changes in order. Write $\mathcal{E}_{\le t}$ for the events up to time t, and $\mathcal{F}_t$ for what they tell us together with the sprint calendar, which is fixed at planning. A sprint s has a start a_s, a planned end b_s and a closure time c_s.

The committed set is the board at the start,

$$C_s = \{\, i : s \in \text{sprint}_i(a_s),\ \gamma_i \le a_s,\ r_i \not< a_s \,\}, \qquad (1)$$

where $\text{sprint}_i(t)$ is the issue's sprint field at time t as replayed from $\mathcal{E}_{\le t}$. The outcomes are read off the end state. With $n_s = |C_s|$ and the carry ratio

$$\rho_s = \frac{|\{i \in C_s : r_i > b_s\}|}{n_s}, \qquad (2)$$

spillover is $y_s^{\text{sp}} = \mathbf{1}[\rho_s > 0]$, serious spillover is $y_s^{\tau} = \mathbf{1}[\rho_s \ge \tau]$, late closure is $y_s^{\text{late}} = \mathbf{1}[c_s - b_s > \delta]$ with δ one day (three for "seriously late"), and scope churn is $\mathbf{1}[m_s/n_s \ge \theta]$, where m_s counts issues added after a_s and still in the sprint at b_s.

3.2 Admissible features

Definition 1 (Admissible feature)**.** *A feature $x(s,t)$ for sprint s at prediction time t is admissible if it is a function of $\mathcal{F}_t$ alone, that is, of $\mathcal{E}_{\le t}$ and the sprint calendar.*

Two common practices break this rule. First, a history feature that summarises an earlier sprint u of the same project is admissible at a_s only if u's outcome was known by then, $\max(b_u, c_u) \le a_s$. A "previous sprint" picked by start date alone breaks this whenever sprints overlap, which happens for 60% of the sprints in our cohort. Second, a field read from the static export is $\mathcal{F}_\infty$-measurable, not $\mathcal{F}_{a_s}$-measurable. Wherever the log records no change for a field, the replay has to fall back on a value that may postdate planning. We record the share of committed issues affected, ϕ_s, and repeat the main comparisons on sprints with $\phi_s = 0$. Issue type, priority and assignee are left out altogether, because the export only holds their final values.

3.3 Empty sprints inflate ROC-AUC

A sprint with $n_s = 0$ cannot spill, so it is a negative by construction. Keeping such sprints in a spillover evaluation rewards the model for something it did not have to learn.

Proposition 1. *Split the negatives of a test set into structural negatives N_0 and ordinary negatives N_1, and let $\pi_0 = |N_0|/(|N_0| + |N_1|)$. Then*

$$\text{AUC}_{\text{all}} = \pi_0\, A_0 + (1 - \pi_0)\, \text{AUC}_{N_1}, \qquad (3)$$

where A_0 is the probability that a random positive outscores a random structural negative, and AUC_{N_1} is the AUC on the positives and ordinary negatives alone. If the model ranks every structural negative below every positive, $A_0 = 1$ and the inflation is $\pi_0(1 - \text{AUC}_{N_1})$.

Proof. AUC is the probability that a random positive outscores a random negative. Conditioning on which kind of negative is drawn gives (3). □

The decomposition is elementary. Its use is that it puts an exact number on the inflation.

On our time-split test set the full population has 588 negatives, and 517 of them commit nothing, so $\pi_0 = 0.879$. The model's AUC on nonempty sprints is 0.882. With $A_0 = 1$, (3) predicts 0.986 for the full population; the observed value is 0.986, and the measured A_0 is 1.000. The gap of 0.104 between the two headline numbers is entirely the empty sprints. Any study that keeps them in the denominator reports this gap as skill.

3.4 Why a fitted threshold does not travel

Youden's index picks the threshold that maximises sensitivity plus specificity, which is the same as minimising FNR + FPR. With prevalence π, a miss cost c_{FN} and a false-alarm cost c_{FP}, the expected cost per sprint is $\pi c_{\text{FN}}\text{FNR} + (1 - \pi)c_{\text{FP}}\text{FPR}$. This is proportional to FNR + FPR exactly when

$$\frac{c_{\text{FN}}}{c_{\text{FP}}} = \frac{1 - \pi}{\pi}. \qquad (4)$$

Proposition 2. *A Youden threshold fitted on a population with prevalence π_{train} is cost-optimal on a population with prevalence π only if the implied cost ratio (4) is the same, that is, only if $\pi = \pi_{\text{train}}$. For a calibrated score the cost-optimal rule warns when $p \ge p^* = c_{\text{FP}}/(c_{\text{FP}} + c_{\text{FN}})$, which does not depend on prevalence [32]. A fixed score threshold carried to a team with lower prevalence therefore acts like a rule that prices a miss at only $(1 - \pi_{\text{train}})/\pi_{\text{train}}$ false alarms, and its sensitivity falls with π.*

The link between a threshold rule and the cost ratio it implies is standard in the ROC literature [31]. What we add is its use to explain why a threshold that worked on TAWOS misses most failures on some new teams.

For late closure the training prevalence is 0.41, so the implied ratio is 1.4. On the 8 testable outside projects the same threshold meets prevalences from 0.07 to 0.59, where (4) would call for ratios from 13 down to 0.7. The observed sensitivity runs from 0.00 to 0.86, with a rank correlation of 0.93 with prevalence. The ranking travels because AUC does not depend on a threshold; the threshold does not travel. A deployment has to recalibrate it per team or set it from stated costs.

3.5 A minimal model of the two spillover labels

Suppose each committed issue of sprint s is finished by the planned end independently with probability q_s, a property of the team and the period rather than of the issue. The

number left unfinished is then Binomial$(n_s, 1-q_s)$, and

$$\Pr[y_s^{\mathrm{sp}} = 1] = 1 - q_s^{n_s}, \tag{5}$$

$$\mathbb{E}[\rho_s] = 1 - q_s, \qquad \mathrm{Var}[\rho_s] = \frac{q_s(1-q_s)}{n_s}. \tag{6}$$

Two predictions follow. By (5) the "any" label is driven by n_s and saturates fast: at the cohort's median carry ratio, 0.71, $q_s \approx 0.29$ and a sprint of three issues already spills with probability 0.976. The median nonempty sprint commits 6 issues, and 90.6% of nonempty sprints spill. So committed scope should carry most of what can be known about plain spillover. By (6) a threshold label on ρ_s is driven by $1 - q_s$, the team's completion rate, with n_s entering only through the variance. Serious spillover should therefore depend on planning-time history and capacity rather than on scope, and the gap between scope-only and history-aware models should be larger for it than for plain spillover. Section 6 tests both.

The model also gives a predictor with no learning at all. Estimate q_s from the team's last finished sprints, which is admissible, and read the risk of either label off the binomial distribution. If this one-number-per-team model ranks sprints nearly as well as a learned model, then most of what the learned model does is estimate q_s.

3.6 When issues do not finish independently

The binomial model assumes that the issues of a sprint finish independently. They plausibly do not: a blocked dependency, a sick developer or an underestimated epic holds back several issues at once. A standard way to allow for this [36] is to let the unfinished probability p_s itself vary from sprint to sprint around $1 - q_s$, with a beta distribution whose spread is set by one parameter $r \in (0, 1)$, the correlation between two issues of the same sprint. The count left unfinished is then beta-binomial.

Proposition 3. *If* $p_s \sim \mathrm{Beta}\big((1-q_s)(1-r)/r,\ q_s(1-r)/r\big)$ *and, given* p_s*, each committed issue is left unfinished independently with probability* p_s*, then* $\mathbb{E}[\rho_s] = 1 - q_s$ *and*

$$\mathrm{Var}[\rho_s] = \frac{q_s(1-q_s)}{n_s}\big(1 + (n_s - 1)\, r\big). \tag{7}$$

As n_s *grows,* $\mathrm{Var}[\rho_s] \to r\, q_s(1-q_s)$ *rather than to zero, and* $\Pr[\rho_s \geq \tau] \to \Pr[p_s \geq \tau]$ *rather than to 0 or 1.*

Proof. By the law of total variance, $\mathrm{Var}[\rho_s] = \mathbb{E}[p_s(1-p_s)]/n_s + \mathrm{Var}[p_s]$. For the beta distribution above, $\mathrm{Var}[p_s] = r\, q_s(1-q_s)$ and $\mathbb{E}[p_s(1-p_s)] = (1-r)\, q_s(1-q_s)$. Adding the two terms gives (7). Given p_s, $\rho_s \to p_s$ as n_s grows, which gives the limits. □

Two things follow. If $r > 0$, the binomial model's variance is too small by the factor $1 + (n_s - 1)r$, which grows with the size of the sprint, so the binomial probabilities will be too extreme for large sprints. And committed scope matters even less for serious spillover than the binomial model says, because a large sprint no longer averages its luck away. Section 6 estimates r and checks both. Neither model covers scope added during the sprint. They are reading aids, not claims about how teams work.

3.7 Warnings as decisions

A warning raised at checkpoint c costs κ and, if the sprint would otherwise fail, averts a fraction $e(c)$ of the failure cost F, with e falling as c grows. Warning at calibrated risk p has expected cost $\kappa + Fp(1 - e(c))$ against Fp for staying silent, so the warning pays when

$$p \geq \frac{\kappa}{e(c)\, F}. \tag{8}$$

The bar rises as the sprint goes on, because a late warning saves less. An in-sprint model is useful only where its gain in ranking outpaces the fall in $e(c)$. Because (8) needs a calibrated p, we report calibration beside AUC.

4 Data and Reconstruction

4.1 Cohort

TAWOS [10] holds Jira data from open-source projects; 36 of them have sprints. We count a sprint as usable if it is closed, has start, end and completion dates, and neither the end nor the completion is before the start. Thirty-two projects have at least 30 usable sprints. Three of them (MongoDB Compass and the MongoDB C++ and Java drivers) are also in our outside test set, which we pulled from MongoDB's live Jira, so we drop them from TAWOS to keep the outside test clean. The cohort is the remaining 29 projects. After dropping each project's first sprint as a cold start, it has 4,235 sprints from February 2012 to October 2020, between 35 and 557 per project (median 97). Of these, 2,671 commit at least one issue; we call them nonempty.

The outside set is the ten projects used in our earlier study: MariaDB MaxScale and ColumnStore, and eight MongoDB projects, exported from the organisations' public Jira and converted to the TAWOS layout. They are used only for testing.

4.2 Reconstruction

For every sprint we replay the Sprint and Story Points fields of every issue back to the sprint's start and form the committed set (1). Issues added later, issues removed later, and their story points at the start all come from the replay. Where an issue has no recorded story-point change, the replay falls back on the exported value; on average 12.5% of a sprint's committed issues are affected. Two custom fields are excluded from the replay: "Actual Story Points" in Spring XD, which is recorded after the work, and a retired duplicate in Moodle. Code, queries

Table 1: Outcomes, populations and positive rates (29 TAWOS projects). Nonempty: sprints that commit at least one issue. Indented rows are sensitivity versions.

Outcome	Population	Sprints	Positive
Spillover	Nonempty	2,671	91%
Serious spillover	Nonempty	2,671	83%
Late closure	All	4,235	41%
Seriously late	All	4,235	24%
Scope churn	Nonempty, 5+	1,593	64%
Serious sp., median	same	2,671	52%
Serious sp., 20%	same	2,671	87%
Serious sp., 50%	same	2,671	73%
Seriously late, 7 days	same	4,235	14%
Scope churn, 50%	same	1,593	48%

and the exact export are in the repository listed under Data Availability.

5 Outcomes, Features and Protocol

5.1 Outcomes

Table 1 lists the five outcomes. Late closure and spillover are carried over from our earlier work. The other three are new.

Serious spillover asks how much was left over, not whether anything was. The primary threshold is a carry ratio of 30%. The median nonempty sprint in TAWOS leaves 71% of its committed issues unfinished, so this label is still 83% positive. We therefore also report a median-based version (carry ratio above the training-set median, 52% positive) and thresholds of 20% and 50%.

Seriously late means the sprint was closed in Jira more than three days after its planned end (seven as a sensitivity check). Like late closure, it records an administrative event, only a larger one.

Scope churn means the issues added after the start and still in the sprint at its planned end make up at least 30% of the committed count (50% as a sensitivity check). It is defined only for sprints with five or more committed issues, because one added issue in a two-issue sprint is 50%. It is predicted at planning time only, because during the sprint the additions so far give the answer away.

5.2 Feature families

Table 2 lists the six families. All features are admissible in the sense of Section 3.

The four *new planning-time features* are: the share of committed issues that had been in another sprint before (carried in); the log of the median age of committed issues at the start; the share of committed issues with no estimate; and the share whose estimate was changed before the start (re-estimated). Each is computed from the replayed log.

Table 2: Feature sets. Planning-time sets are scored at the sprint start; in-sprint sets at 25% and 50% of the planned duration.

Set	Contents
scope	Committed issues, committed story points, sprint length.
strict	scope, parallel sprints, the last three finished sprints (completion ratio, carry-over, delivered points, completed and committed issues), scope relative to capacity, past lateness, and the project.
strict+new	strict and the four new features below.
choet	Iteration features of [1] that the export supports: duration, issues and points at start, added and removed since start, done and to do at the checkpoint.
worki	Burndown-shape indicators R1 to R11 after [4], on issue counts.
ours	strict+new and 13 progress counts: elapsed share, days left, done and remaining issues and points, burn gap, required and observed rate, added and removed ratios.
ours+worki	ours and R1 to R11.

The *worki* set follows our reading of the verbal definitions in [4], computed on the daily burndown of issue counts up to the checkpoint. Their study graded finished sprints; we score the same kind of indicator during the sprint, which is a different task. The *choet* set leaves out what a public export cannot support: in-progress counts (there is no status history), team size and the Scrum master (the assignee is a final value), and all issue-level features. At planning time their supported set reduces to our scope set. Neither comparison therefore ranks the two studies; each measures a family of signal.

One asymmetry matters for reading the results. The strict and ours sets include the project as a categorical feature, as in our earlier work; scope, choet and worki do not. On the time split, part of a strict or ours gain may therefore be project identity. Leave-one-project-out testing removes the project from every set, and we report it beside every primary result.

5.3 Models and protocol

We use logistic regression with balanced class weights, a random forest [37] (300 trees, depth 5) and histogram gradient boosting [38] (depth 3, learning rate 0.05, 200 iterations), all from scikit-learn [39] with fixed settings and seed 42. There is no hyperparameter search.

The *primary analysis* is gradient boosting on a per-project time split: the first 70% of each project's sprints by start date for training, the rest for testing (1,288 test sprints, 814 of them nonempty). For each outcome and each comparison we report one paired AUC difference with two 95% bootstrap intervals from 2,000 resamples: one

that resamples whole projects [40], which is the test, and one that resamples sprints, which is reported beside it. Everything else is secondary and read as a pattern: the other two models, leave-one-project-out testing, five time-ordered repeats (training on the first 40% to 80% of each project and testing on the next 20%), and the outside projects (trained on all of TAWOS, without the project feature and without story points, which MariaDB does not record). Decision thresholds are fixed by Youden's index [41] on out-of-fold training scores, never on the test set.

5.4 Hypotheses

We wrote the study plan, the primary analysis and four hypotheses into a dated plan before the full-cohort run, and logged every later change with its reason. A hypothesis holds only where the project-level interval is above zero.

- **H1** (new features, strict+new minus strict, planning time): positive for scope churn and serious spillover, small and positive for spillover, about zero for the two late labels.
- **H2** (burndown shape, ours minus worki, spillover labels): positive at both checkpoints.
- **H3** (planning context on top of progress counts, ours minus choet): positive and below 0.05 for every outcome.
- **H4** (new labels can be ranked, strict+new minus scope): positive for seriously late and scope churn, about zero for serious spillover.

The plan was written after a development run on eight of the 29 projects, which hold 2,602 of the 4,235 sprints. The other 21 projects had not been looked at when the hypotheses were written. We therefore repeat every hypothesis test on those 21 projects alone, training and testing on them only, as a confirmation set (Section 6.6).

6 Results

6.1 Planning time

Table 3 gives the AUC of the three planning-time sets and Table 4 the primary differences.

H1 holds for the two spillover labels. The new features add 0.028 [0.007, 0.047] for spillover and 0.026 [0.004, 0.050] for serious spillover. The gain has the same sign for all three models and under leave-one-project-out testing (0.020 [0.006, 0.037] and 0.027 [0.009, 0.048]). For the two late labels the difference is about zero, as predicted. For scope churn it is 0.042, but the project interval just crosses zero, so H1 does not hold there.

H4 fails where it was most specific. We predicted that history would add nothing to serious spillover beyond scope. It adds 0.104 [0.036, 0.179] on the time split and 0.168 [0.111, 0.223] under leave-one-project-out testing, the largest planning-time gain in the study. Seriously late (0.033) and scope churn (0.088) are positive on the sprint-level interval only, so H4 does not hold for either.

Late closure is hard. Scope alone reaches 0.718, and nothing at planning time does much better.

Table 3: Planning-time ROC-AUC, gradient boosting. Time: per-project time split. LOPO: leave one project out (no project feature).

	Time split			LOPO		
Outcome	scope	strict	+new	scope	strict	+new
Spillover	0.825	0.837	0.865	0.803	0.835	0.855
Serious spillover	0.671	0.749	0.775	0.586	0.727	0.754
Late closure	0.718	0.735	0.729	0.659	0.729	0.726
Seriously late	0.663	0.697	0.697	0.667	0.702	0.700
Scope churn	0.544	0.590	0.632	0.539	0.558	0.576

Table 4: Primary planning-time differences in AUC (gradient boosting, time split) with 95% bootstrap intervals over projects and over sprints. H1: strict+new minus strict. H4: strict+new minus scope.

	Outcome	Δ	Projects	Sprints
H1	Spillover	+0.028	[+0.007, +0.047]	[+0.008, +0.048]
H1	Serious spillover	+0.026	[+0.004, +0.050]	[+0.004, +0.046]
H1	Late closure	−0.006	[−0.016, +0.004]	[−0.011, −0.001]
H1	Seriously late	0.000	[−0.009, +0.006]	[−0.005, +0.005]
H1	Scope churn	+0.042	[−0.003, +0.084]	[+0.007, +0.076]
H4	Spillover	+0.040	[+0.003, +0.082]	[+0.006, +0.072]
H4	Serious spillover	+0.104	[+0.036, +0.179]	[+0.049, +0.161]
H4	Late closure	+0.011	[−0.033, +0.057]	[−0.016, +0.038]
H4	Seriously late	+0.033	[−0.015, +0.088]	[+0.003, +0.064]
H4	Scope churn	+0.088	[−0.016, +0.222]	[+0.016, +0.157]

6.2 The binomial model's prediction

Section 3 predicts that history should beat scope by more for serious spillover than for plain spillover. It does. On the time split, strict minus scope is 0.012 for spillover against 0.079 for serious spillover; under leave-one-project-out testing it is 0.032 against 0.141. Plain spillover is mostly a matter of how many issues were committed, and serious spillover is mostly a matter of which team committed them. The prediction contradicts H4, which we wrote on the same day; the data side with the model.

6.3 During the sprint

Tables 5 and 6 give the in-sprint results. All sets are scored on the nonempty sprints still open at the checkpoint (810 test sprints at 25%, 803 at 50%), so late closure here has a different population from Table 3.

H2 holds. Our set beats the burndown-shape set by 0.118 to 0.223 AUC on both spillover labels at both check-

Table 5: In-sprint ROC-AUC at 25% and 50% of the planned duration, gradient boosting, time split.

Outcome	At	scope	worki	choet	ours	+worki
Spillover	25%	0.827	0.803	0.870	0.922	0.920
	50%	0.823	0.720	0.897	0.943	0.942
Serious spillover	25%	0.688	0.684	0.785	0.851	0.847
	50%	0.681	0.713	0.824	0.891	0.888
Late closure	25%	0.687	0.585	0.661	0.718	0.717
	50%	0.682	0.635	0.679	0.722	0.724
Seriously late	25%	0.615	0.520	0.617	0.653	0.654
	50%	0.624	0.556	0.618	0.656	0.668

points. The burndown set is not just weaker than ours. For late closure at 25%, for seriously late at both checkpoints and for spillover at 50%, it ranks worse than committed scope alone, with intervals below zero, and adding it to our set changes nothing (every ours+worki minus ours difference is within 0.012).

H3 holds in direction, not in size. Planning context adds to progress counts for every outcome, and the project interval clears zero for both spillover labels (0.046 to 0.067). For serious spillover the gain is above the 0.05 we expected. Because ours includes the project feature and choet does not, we checked the same comparison under leave-one-project-out testing, where neither set has it: the gain stays at 0.036 [0.012, 0.065] and 0.028 [0.008, 0.052] for spillover, and 0.078 [0.053, 0.106] and 0.064 [0.039, 0.092] for serious spillover. For the two late labels the project intervals include zero.

The spillover labels are where the signal is. Serious spillover goes from 0.775 at planning time to 0.851 at 25% and 0.891 at 50%. For late closure our set reaches only 0.718 and 0.722, against 0.687 and 0.682 for scope alone on the same sprints.

6.4 Outside projects

Table 7 trains on all of TAWOS and tests on each outside project with at least five sprints in each class. Nothing transfers reliably. For the two spillover labels, where history helped most inside TAWOS, it does not help outside: scope alone scores 0.785 for spillover and 0.592 for serious spillover, and the history-aware sets score lower. For late closure all three sets sit near 0.65. The spread across projects is about ± 0.1 to ± 0.2. This fits the reading in Section 3: history features estimate one team's completion rate, and that estimate does not carry to another organisation's teams.

6.5 Calibration and decisions

Table 8 gives the Brier score, the Brier score of always predicting the test prevalence, the expected calibration error over ten bins, and the calibration slope. For the spillover labels the models are well calibrated (slopes 0.88 to 1.04) and clearly beat the reference. For the late labels they are overconfident (slopes 0.55 to 0.85), and for seriously late during the sprint the Brier score is no better than the reference. By (8), an in-sprint lateness warning built on these scores would rest on probabilities that are not trustworthy.

With a Youden threshold from the training folds, the planning-time model flags serious spillover with sensitivity 0.73 at specificity 0.69, spillover with 0.68 at 0.85, and late closure with 0.62 at 0.76.

6.6 Confirmation on the untouched projects

Table 9 repeats every hypothesis comparison on the 21 projects that played no part in writing the hypotheses (502 test sprints, 303 of them nonempty). The in-sprint results replicate. H2 holds in all four comparisons (0.141 to 0.260), and H3 holds for both spillover labels at both checkpoints (0.060 to 0.109), while for the two late labels its intervals again include zero. At planning time, the gain from the new features replicates for serious spillover (0.033 [0.004, 0.078]) but not for spillover (0.010 [−0.014, 0.036]), so we treat the spillover part of H1 as unconfirmed. History beats scope for both spillover labels (0.096 and 0.178) and, as in the full cohort, not clearly for the other labels. The confirmation set is smaller and its intervals are wider. Where a sign changes (scope churn in H1), the result did not hold in the full cohort either.

6.7 A model with no learning

Table 10 scores the binomial model of Section 3 as a predictor. For each sprint, q_s is the pooled completion rate of the team's last three finished sprints known at the start, or the training-set rate where there is no history. Nothing else is fitted. Its ranking comes close to gradient boosting: 0.858 against 0.865 for spillover (difference 0.007 [−0.018, 0.043]) and 0.752 against 0.775 for serious spillover (0.023 [−0.017, 0.072]). For serious spillover it beats scope alone by 0.081 [0.005, 0.144]. Most of what the learned model adds over scope at planning time is therefore an estimate of one number per team.

Its probabilities are another matter. The calibration slopes are 0.40 and 0.25, so the binomial model is far too sure of itself. Table 11 shows why. In every band of sprint size the observed variance of the carry ratio is larger than the binomial variance, by a factor of 1.7 for sprints of one to three issues and 5.8 for sprints of 16 or more. A gap that grows with sprint size is what Proposition 3 predicts when $r > 0$. The ratio of observed to binomial variance over the training sprints, the quantity behind Tarone's test of binomial fit [42], is 4.5, and the maximum-likelihood estimate is $r = 0.18$ (likelihood-ratio statistic 2,245 against $r = 0$). The beta-binomial model matches the observed variance for large sprints (0.041 against 0.044) and raises the slopes to 0.58 and 0.46, at a small cost in ranking (0.010 and 0.013 AUC). Neither is as well calibrated as

Table 6: Primary in-sprint differences in AUC (gradient boosting, time split) with 95% bootstrap intervals over projects and over sprints. The worki minus scope rows are not a hypothesis and are given for reading.

	Comparison	Outcome	At	Δ	Projects	Sprints
H2	ours − worki	Spillover	25%	+0.118	[+0.069, +0.175]	[+0.069, +0.171]
H2	ours − worki	Spillover	50%	+0.223	[+0.145, +0.297]	[+0.161, +0.288]
H2	ours − worki	Serious spillover	25%	+0.167	[+0.104, +0.248]	[+0.109, +0.227]
H2	ours − worki	Serious spillover	50%	+0.178	[+0.098, +0.251]	[+0.123, +0.233]
H3	ours − choet	Spillover	25%	+0.051	[+0.020, +0.083]	[+0.018, +0.082]
H3	ours − choet	Spillover	50%	+0.046	[+0.011, +0.088]	[+0.014, +0.076]
H3	ours − choet	Serious spillover	25%	+0.066	[+0.025, +0.119]	[+0.023, +0.107]
H3	ours − choet	Serious spillover	50%	+0.067	[+0.017, +0.120]	[+0.027, +0.107]
H3	ours − choet	Late closure	25%	+0.057	[−0.001, +0.123]	[+0.018, +0.094]
H3	ours − choet	Late closure	50%	+0.043	[−0.012, +0.112]	[+0.007, +0.079]
H3	ours − choet	Seriously late	25%	+0.036	[−0.035, +0.111]	[−0.008, +0.078]
H3	ours − choet	Seriously late	50%	+0.038	[−0.043, +0.133]	[−0.007, +0.088]
–	worki − scope	Spillover	25%	−0.023	[−0.095, +0.048]	[−0.085, +0.040]
–	worki − scope	Spillover	50%	−0.102	[−0.189, −0.016]	[−0.177, −0.028]
–	worki − scope	Serious spillover	25%	−0.004	[−0.101, +0.085]	[−0.079, +0.070]
–	worki − scope	Serious spillover	50%	+0.031	[−0.056, +0.118]	[−0.056, +0.111]
–	worki − scope	Late closure	25%	−0.102	[−0.160, −0.038]	[−0.147, −0.055]
–	worki − scope	Late closure	50%	−0.047	[−0.103, +0.012]	[−0.092, −0.003]
–	worki − scope	Seriously late	25%	−0.095	[−0.158, −0.040]	[−0.151, −0.042]
–	worki − scope	Seriously late	50%	−0.068	[−0.130, −0.007]	[−0.122, −0.016]

Table 7: Outside projects: mean ± s.d. of per-project AUC, gradient boosting trained on all TAWOS sprints. k: projects with at least five sprints in each class.

Outcome	k	scope	strict	strict+new
Spillover	4	0.785 ± 0.119	0.734 ± 0.160	0.729 ± 0.170
Serious spillover	9	0.592 ± 0.125	0.546 ± 0.109	0.573 ± 0.110
Late closure	8	0.655 ± 0.148	0.654 ± 0.091	0.651 ± 0.085
Seriously late	8	0.571 ± 0.172	0.610 ± 0.200	0.617 ± 0.224
Scope churn	5	0.425 ± 0.204	0.589 ± 0.090	0.593 ± 0.115

Table 8: Calibration on the time-split test set, gradient boosting. Ref.: Brier score of the test prevalence. ECE: expected calibration error, ten bins.

Outcome	Stage	Brier	Ref.	ECE	Slope
Spillover	planning	0.066	0.087	0.025	0.95
	in-sprint 25%	0.050	0.087	0.015	1.04
	in-sprint 50%	0.040	0.087	0.010	1.03
Serious spillover	planning	0.103	0.122	0.023	0.88
	in-sprint 25%	0.083	0.122	0.030	0.91
	in-sprint 50%	0.066	0.121	0.009	0.93
Late closure	planning	0.197	0.230	0.043	0.85
	in-sprint 25%	0.201	0.226	0.067	0.73
	in-sprint 50%	0.200	0.227	0.066	0.73
Seriously late	planning	0.148	0.162	0.044	0.79
	in-sprint 25%	0.146	0.149	0.054	0.59
	in-sprint 50%	0.149	0.150	0.039	0.55

gradient boosting (slopes 0.95 and 0.88). The learning is needed mostly for trustworthy probabilities, not for the ranking.

6.8 Further checks

Provenance. On the 422 nonempty test sprints with no story-point fallback, the new features add 0.023 [−0.007, 0.051] for spillover and 0.027 [−0.001, 0.053] for serious spillover: the same size as the primary result, with intervals that just touch zero. The gain is not an artefact of fallback values, but this subset alone would not establish it.

Other thresholds. The median-based serious-spillover label behaves like the primary one: strict+new scores 0.720 against 0.572 for scope, and the new features add 0.032 [0.007, 0.056]. At the 50% threshold they add 0.032 [0.006, 0.061] and at 20% 0.036 [0.019, 0.064]. For scope churn at 50% they add 0.054 [0.015, 0.087], so the primary scope-churn result may be a near miss rather than a null.

Repeats. The five time-ordered repeats agree with the time split: strict+new scores 0.860 ± 0.016 for spillover, 0.769 ± 0.022 for serious spillover and 0.728 ± 0.013 for late closure.

6.9 Summary of hypotheses

Fig. 1 puts all 22 comparisons on one scale. H1 holds for spillover and serious spillover and is about zero for lateness, as predicted, but misses for scope churn. H2 holds. H3 holds in direction for every outcome and clears zero

Table 9: Every hypothesis comparison on the full cohort and on the confirmation set of 21 projects not used in the development run (gradient boosting, time split). ✓: project-level interval above zero.

		29 projects		21 untouched projects		
	Outcome	Δ		Δ	Projects	
H1	Late closure	−0.006		−0.007	[−0.021, +0.007]	
H1	Seriously late	0.000		+0.009	[−0.003, +0.020]	
H1	Spillover	+0.028	✓	+0.010	[−0.014, +0.036]	
H1	Serious spillover	+0.026	✓	+0.033	[+0.004, +0.078]	✓
H1	Scope churn	+0.042		−0.013	[−0.066, +0.045]	
H4	Late closure	+0.011		+0.010	[−0.050, +0.066]	
H4	Seriously late	+0.033		+0.042	[−0.019, +0.085]	
H4	Spillover	+0.040	✓	+0.096	[+0.053, +0.146]	✓
H4	Serious spillover	+0.104	✓	+0.178	[+0.095, +0.270]	✓
H4	Scope churn	+0.088		+0.064	[−0.114, +0.209]	
H2	Spillover, 25%	+0.118	✓	+0.141	[+0.053, +0.232]	✓
H2	Spillover, 50%	+0.223	✓	+0.260	[+0.142, +0.375]	✓
H2	Serious spillover, 25%	+0.167	✓	+0.258	[+0.180, +0.350]	✓
H2	Serious spillover, 50%	+0.178	✓	+0.206	[+0.118, +0.278]	✓
H3	Spillover, 25%	+0.051	✓	+0.060	[+0.018, +0.106]	✓
H3	Spillover, 50%	+0.046	✓	+0.077	[+0.017, +0.144]	✓
H3	Serious spillover, 25%	+0.066	✓	+0.109	[+0.053, +0.185]	✓
H3	Serious spillover, 50%	+0.067	✓	+0.091	[+0.040, +0.152]	✓
H3	Late closure, 25%	+0.057		+0.023	[−0.037, +0.076]	
H3	Late closure, 50%	+0.043		+0.011	[−0.044, +0.063]	
H3	Seriously late, 25%	+0.036		+0.057	[−0.010, +0.118]	
H3	Seriously late, 50%	+0.038		+0.003	[−0.059, +0.073]	

Table 10: The binomial model as a predictor on the nonempty test sprints (814). AUC and calibration slope (1 is ideal).

	Spillover		Serious sp.	
Model	AUC	Slope	AUC	Slope
Gradient boosting, scope	0.825	0.97	0.671	0.85
Binomial, no learning	0.858	0.40	0.752	0.25
Beta-binomial, one parameter	0.847	0.58	0.739	0.46
Gradient boosting, strict+new	0.865	0.95	0.775	0.88

for the spillover labels, with a larger gain than predicted for serious spillover. H4 fails: serious spillover, predicted to be unrankable beyond scope, gives the largest gain, and the other two new labels pass only on the sprint-level interval. On the 21 untouched projects, H2, H3 for the spillover labels, H1 for serious spillover and the serious-spillover gain over scope all replicate; H1 for spillover does not.

7 Discussion

7.1 What a team can take from this

Three things. First, during a sprint, simple progress counts (how much is done, how much is left, how fast it is going) carry nearly all of the signal, and their warnings for leftover work are calibrated. Burndown-shape indicators add nothing on top of them and, for lateness, do worse than knowing the sprint's size. Second, before a sprint starts, the team's own recent history is what says how much will be left over. Most of it is one number, the share of committed issues the team finished in its last few sprints, and a team can use that number directly through the binomial model; the four new features add a little to it. Third, none of this travels. A model trained on other organisations' teams is no better than a size rule, and a threshold carried from one team to another changes what a miss is worth (Proposition 2). A team that wants these warnings has to fit them on its own history.

Table 11: Variance of the carry ratio around $1-\hat{q}_s$ on the training sprints, by committed issues: observed, binomial (6) and beta-binomial (7) with $r = 0.18$.

Issues	Sprints	Observed	Binomial	Beta-bin.
1–3	568	0.181	0.108	0.117
4–7	420	0.087	0.035	0.061
8–15	432	0.062	0.016	0.045
16+	437	0.044	0.008	0.041

7.2 Why lateness is hard

Late closure is the moment someone pressed "Complete sprint" in Jira. That depends on meetings, holidays and habit as much as on the work, and no feature family we tried moves it much. We keep it for continuity, but we would not build a tool on it. Seriously late, the larger version of the same event, is no easier to rank, and its scores are the worst calibrated in the study.

7.3 Threats to validity

Construct. The labels come from Jira fields, not from asking teams whether a sprint went badly. Late closure and seriously late measure when a sprint was closed in the tool. Spillover counts issues, not effort.

Internal. The reconstruction depends on the change log. Where it is missing, story points fall back on the exported value; we measured the share and repeated the main comparisons without it. Issue type, priority and assignee are left out because only their final values exist, which also means the choet set lacks team and issue features that its authors used. The strict and ours sets carry the project as a feature; we repeated the main comparisons without it.

Hypotheses written after a look at the data. The plan, the primary analysis and the hypotheses were written before the full run but after a development run on eight of the 29 projects, which hold 2,602 of the 4,235 sprints. Some choices were made after that look, most clearly the median-based serious-spillover label, and all of them are logged with dates. The hypotheses are therefore not independent of 8 of the 29 projects. The confirmation set answers most of this concern: the in-sprint hypotheses and the serious-spillover results hold on the 21 projects

the development run never saw, while the spillover part of H1 does not, and we report it as unconfirmed. The hypotheses that failed (H4, the size in H3) also failed against what the development run had suggested, and the outside projects were never used for any decision.

Conclusion. The four hypotheses rest on 22 paired comparisons, repeated on the confirmation set, with no correction for multiple testing. The project-level interval resamples 29 clusters, which is coarse; we report the sprint-level interval beside it and treat a result as holding only when the project interval clears zero.

External. All projects are open source, and a handful of organisations (Atlassian, Appcelerator and a few others) contribute most TAWOS projects. The outside test covers two more organisations and ten projects, few of which have enough sprints in both classes for every label.

8 Conclusion

Under one leakage-aware protocol, the kinds of signal used in the sprint-risk literature differ sharply. Progress counts dominate during a sprint, burndown shape adds nothing, team history explains how much work is left over but not when the sprint is closed, and nothing learned from one organisation's history helps another's. The formal setting turns these observations into results that can be checked: empty sprints inflate AUC by an exact amount, a fitted threshold carries a hidden cost ratio, the choice between "any" and "serious" spillover decides whether history can matter, and issues that do not finish independently explain why a model with no learning ranks well but cannot give trustworthy probabilities. The most useful outcome we added is serious spillover in its median-based form. The obvious next step is to ask teams whether the sprints these models flag are the ones they would have wanted to hear about.

Data Availability

Code, export queries, the reconstructed frames, every result file and the dated study plan with its change log are at `https://github.com/hani-ziad/journal-ieee-access-2027`. TAWOS is available from its authors [10].

References

[1] M. Choetkiertikul, H. K. Dam, T. Tran, A. Ghose, and J. Grundy, "Predicting delivery capability in iterative software development," *IEEE Trans. Softw. Eng.*, vol. 44, no. 6, pp. 551–573, Jun. 2018, doi: 10.1109/TSE.2017.2693989.

[2] M. Choetkiertikul, P. Banyongrakkul, C. Ragkhitwetsagul, S. Tuarob, H. K. Dam, and T. Sunetnanta, "Sprint2Vec: A deep characterization of sprints in iterative software development," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 220–242, Jan. 2025, doi: 10.1109/TSE.2024.3509016.

[3] P. Hearty, N. Fenton, D. Marquez, and M. Neil, "Predicting project velocity in XP using a learning dynamic Bayesian network model," *IEEE Trans. Softw. Eng.*, vol. 35, no. 1, pp. 124–137, Jan. 2009, doi: 10.1109/TSE.2008.76.

[4] Y. J. Pérez Castillo, S. D. Orantes Jiménez, and P. O. Letelier Torres, "Sprint management in Agile approach: Progress and velocity evaluation applying machine learning," *Information*, vol. 15, no. 11, Art. no. 726, 2024, doi: 10.3390/info15110726.

[5] P. G. Obike, V. E. Ekong, and O. U. Obot, "A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning," *Int. J. Comput. Appl.*, vol. 187, no. 49, pp. 40–53, 2025, doi: 10.5120/ijca2025925837.

[6] E. Kula, E. Greuter, A. van Deursen, and G. Gousios, "Factors affecting on-time delivery in large-scale agile software development," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3573–3592, Sep. 2022, doi: 10.1109/TSE.2021.3101192.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012, doi: 10.1145/2382577.2382579.

[8] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[9] H. AbuSharkh, M. Hanif, and R. Damseh, "Reconstructing sprint-start Jira state for leakage-aware outcome prediction," submitted to the IEEE Canadian Conf. Electr. Comput. Eng. (CCECE), 2027.

[10] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, "A versatile dataset of agile open source software projects," in *Proc. 19th Int. Conf. Mining Softw. Repositories (MSR)*, 2022, pp. 707–711, doi: 10.1145/3524842.3528029.

[11] M. Choetkiertikul, H. K. Dam, T. Tran, and A. Ghose, "Characterization and prediction of issue-related risks in software projects," in *Proc. IEEE/ACM 12th Working Conf. Mining Softw. Repositories (MSR)*, 2015, pp. 280–291, doi: 10.1109/MSR.2015.33.

[12] M. Choetkiertikul, H. K. Dam, T. Tran, and A. Ghose, "Predicting the delay of issues with due dates in software projects," *Empir. Softw. Eng.*, vol. 22, no. 3, pp. 1223–1263, 2017, doi: 10.1007/s10664-016-9496-7.

[13] E. Kula, A. van Deursen, and G. Gousios, "Modeling team dynamics for the characterization and prediction of delays in user stories," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2021, pp. 991–1002, doi: 10.1109/ASE51524.2021.9678939.

[14] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for estimating story points," *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 637–656, Jul. 2019, doi: 10.1109/TSE.2018.2792473.

[15] S. Porru, A. Murgia, S. Demeyer, M. Marchesi, and R. Tonelli, "Estimating story points from issue reports," in *Proc. 12th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE)*, 2016, pp. 1–10, doi: 10.1145/2972958.2972959.

[16] M. Fu and C. Tantithamthavorn, "GPT2SP: A transformer-based agile story point estimation approach," *IEEE Trans. Softw. Eng.*, vol. 49, no. 2, pp. 611–625, Feb. 2023, doi: 10.1109/TSE.2022.3158252.

[17] V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2677–2697, Apr. 2023, doi: 10.1109/TSE.2022.3228739.

[18] M. Ortu, G. Destefanis, B. Adams, A. Murgia, M. Marchesi, and R. Tonelli, "The JIRA repository dataset: Understanding social aspects of software development," in *Proc. 11th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE)*, 2015, pp. 1–4, doi: 10.1145/2810146.2810147.

[19] L. Montgomery, C. Lüders, and W. Maalej, "An alternative issue tracking dataset of public Jira repositories," in *Proc. 19th Int. Conf. Mining Softw. Repositories (MSR)*, 2022, pp. 73–77, doi: 10.1145/3524842.3528486.

[20] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 43, no. 1, pp. 1–18, Jan. 2017, doi: 10.1109/TSE.2016.2584050.

[21] A. A. Bangash, H. Sahar, A. Hindle, and K. Ali, "On the time-based conclusion stability of cross-project defect prediction models," *Empir. Softw. Eng.*, vol. 25, no. 6, pp. 5047–5083, 2020, doi: 10.1007/s10664-020-09878-9.

[22] D. Falessi, J. Huang, L. Narayana, J. F. Thai, and B. Turhan, "On the need of preserving order of data when validating within-project defect classifiers," *Empir. Softw. Eng.*, vol. 25, no. 6, pp. 4805–4830, 2020, doi: 10.1007/s10664-020-09868-x.

[23] M. Jimenez, R. Rwemalika, M. Papadakis, F. Sarro, Y. Le Traon, and M. Harman, "The importance of accounting for real-world labelling when predicting software vulnerabilities," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2019, pp. 695–705, doi: 10.1145/3338906.3338941.

[24] M. Shepperd, D. Bowes, and T. Hall, "Researcher bias: The use of machine learning in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 40, no. 6, pp. 603–616, Jun. 2014, doi: 10.1109/TSE.2014.2322358.

[25] A. E. Márquez-Chamorro, M. Resinas, and A. Ruiz-Cortés, "Predictive monitoring of business processes: A survey," *IEEE Trans. Serv. Comput.*, vol. 11, no. 6, pp. 962–977, 2018, doi: 10.1109/TSC.2017.2772256.

[26] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, "Outcome-oriented predictive process monitoring: Review and benchmark," *ACM Trans. Knowl. Discov. Data*, vol. 13, no. 2, pp. 1–57, 2019, doi: 10.1145/3301300.

[27] C. Di Francescomarino, C. Ghidini, F. M. Maggi, and F. Milani, "Predictive process monitoring methods: Which one suits me best?" in *Business Process Management (BPM)*, Lecture Notes in Computer Science, 2018, pp. 462–479, doi: 10.1007/978-3-319-98648-7_27.

[28] S. A. Fahrenkrog-Petersen, N. Tax, I. Teinemaa, M. Dumas, M. de Leoni, F. M. Maggi, and M. Weidlich, "Fire now, fire later: Alarm-based systems for prescriptive process monitoring," *Knowl. Inf. Syst.*, vol. 64, no. 2, pp. 559–587, 2022, doi: 10.1007/s10115-021-01633-w.

[29] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[30] D. J. Hand, "Measuring classifier performance: A coherent alternative to the area under the ROC curve," *Mach. Learn.*, vol. 77, no. 1, pp. 103–123, 2009, doi: 10.1007/s10994-009-5119-5.

[31] J. Hernández-Orallo, P. Flach, and C. Ferri, "A unified view of performance metrics: Translating threshold choice into expected classification loss," *J. Mach. Learn. Res.*, vol. 13, no. 91, pp. 2813–2869, 2012.

[32] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2001, pp. 973–978.

[33] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Rev.*, vol. 78, no. 1, pp. 1–3, 1950, doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[34] B. Van Calster, D. J. McLernon, M. van Smeden, L. Wynants, and E. W. Steyerberg, "Calibration: The Achilles heel of predictive analytics," *BMC Med.*, vol. 17, 2019, doi: 10.1186/s12916-019-1466-7.

[35] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.

[36] J. G. Skellam, "A probability distribution derived from the binomial distribution by regarding the probability of success as variable between the sets of trials," *J. R. Stat. Soc. Ser. B*, vol. 10, no. 2, pp. 257–261, 1948, doi: 10.1111/j.2517-6161.1948.tb00014.x.

[37] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[38] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[39] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, no. 85, pp. 2825–2830, 2011.

[40] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *J. R. Stat. Soc. Ser. B*, vol. 69, no. 3, pp. 369–390, 2007, doi: 10.1111/j.1467-9868.2007.00593.x.

[41] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950, doi: 10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3.

[42] R. E. Tarone, "Testing the goodness of fit of the binomial distribution," *Biometrika*, vol. 66, no. 3, pp. 585–590, 1979, doi: 10.1093/biomet/66.3.585.

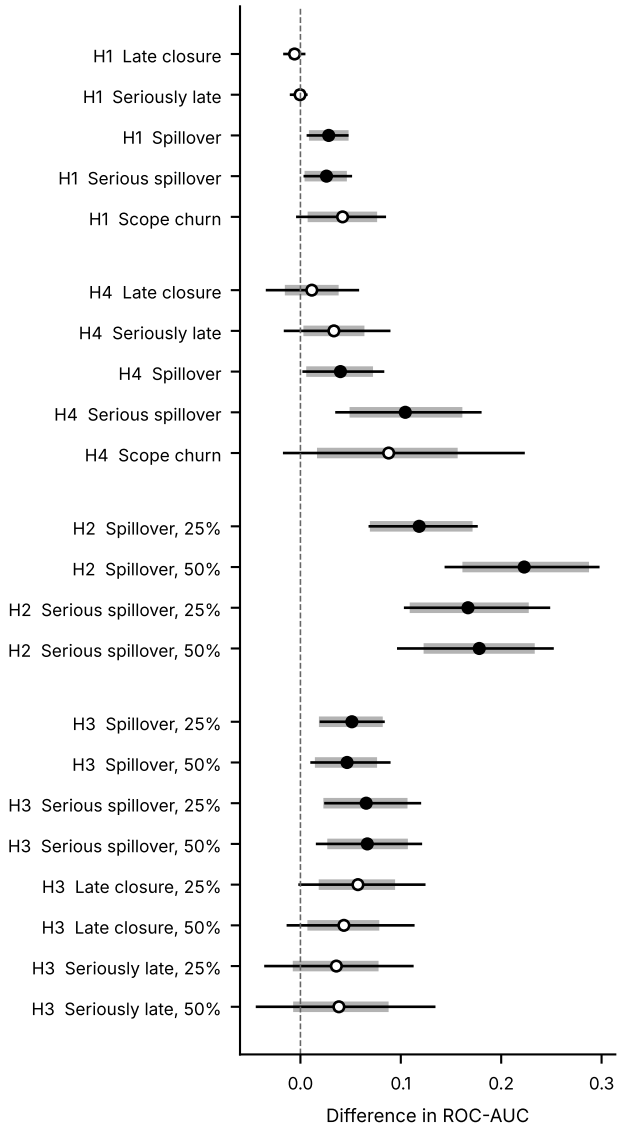


Figure 1: Every hypothesis comparison in the primary analysis. Dot: difference in AUC (filled where the project-level interval is above zero, the rule for a hypothesis to hold). Black line: 95% interval over projects. Grey bar: 95% interval over sprints.